

TP5 – Domaines d'horloge

Questions

1. A l'aide du module LED_driver du TP4, créez une architecture RTL permettant de piloter les deux LED RGB. Les LEDs RGB devront clignoter 10 fois en rouge puis 10 fois en bleu et 10 fois en vert avant de recommencer à partir du rouge. En entrée des modules LED_driver le signal update ne devra pas être à 1 pendant plus d'un coup d'horloge. Il doit s'agir d'une impulsion. Le compteur de temporisation devra compter 100 000 000 coups d'horloge (pour une fréquence d'horloge à 100MHz, cela correspond à 1s). Dans la suite de ce TP, la valeur du compteur de temporisation ne sera pas modifiée, même lorsque les fréquences d'horloges changeront.

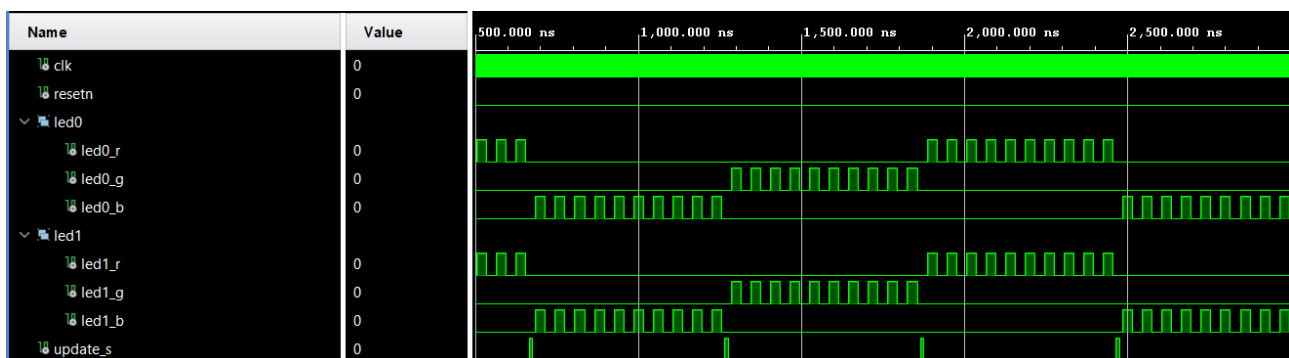
Fait

2. Rédigez le code VHDL correspondant à votre architecture.

Fait

3. Rédigez le testbench et simulez votre design. Vérifiez que les modules réceptionnent correctement le signal update.

Dans le fichier tb_top_single_clock :



On remarque bien que les LEDs RGB clignotent 10 fois en rouge puis 10 fois en bleu et 10 fois en vert avant de recommencer à partir du rouge. De plus, le signal update_s est à 1 pendant un seul coup d'horloge, il doit s'agir bien d'une impulsion.

4. Modifiez votre design pour gérer deux signaux d'horloge différents. La première horloge, clkA, est associé à la logique en dehors des modules LED_driver et au module LED_driver de la LED0. La deuxième horloge, clkB, est associé au module LED_driver de la LED1. Le changement de couleur des deux LEDs à lieu lorsque la LED0 à clignoté 10 fois.

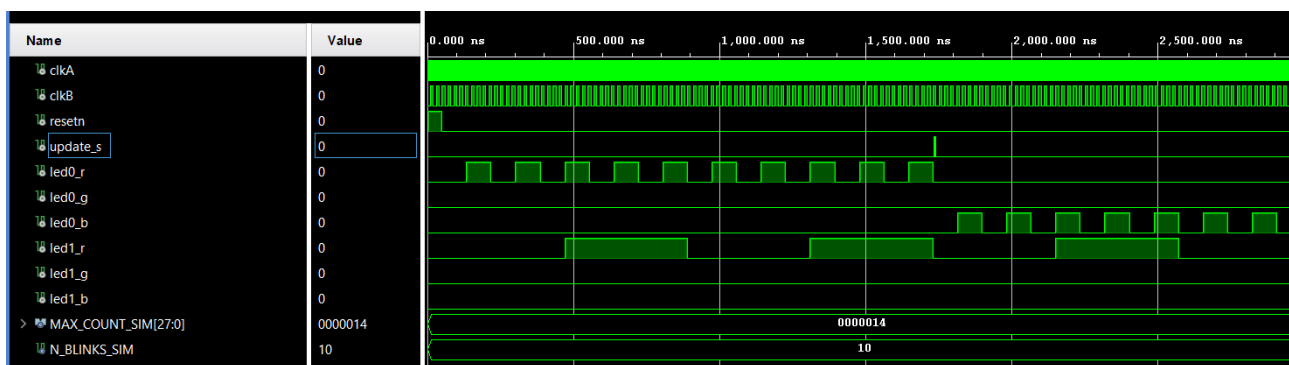
Fait

5. Modifiez votre testbench tel que l'horloge clkA ait une fréquence de 250Mz et clkB 50MHz

Fait

6. Lancer une simulation. Que se passe-t-il au niveau des signaux update des modules LED_driver ? Qu'elle incidence cela a-t-il sur les LEDs ?

Dans le fichier tb_top_dual_clock :



J'observe que led0_r clignote régulièrement puis led_b clignote régulièrement à son tour vers 1800ns, cohérent avec notre FSM qui fonctionne normalement côté clkA.

Donc led1_r clignote bien, mais 5 fois plus lentement en temps réel que led0_r car il fonctionne à un fréquence 5 fois plus lente ce qui est cohérent.

On a cependant une grosse incohérence entre les deux LEDs, en effet, autour de 2000-2500ns, led0_b est en phase de clignotement alors que led1_r est actif au même moment. Les deux LEDs ne sont plus sur la même couleur au même instant, il y a une désynchronisation.

Le problème vient de la réception du signal update_s coté clkB : en effet, une impulsion large de 4 ns (période de clkA) tombe « au hasard » par rapport aux fronts de clkB (période 20 ns) : parfois un front de clkB arrive pendant ces 4 ns et capture l'impulsion correctement, parfois aucun front ne tombe dans cette fenêtre et l'impulsion est simplement « perdue » pour ce domaine.

C'est cette réception aléatoire qui explique cette désynchronisation.

7. Proposez une solution pour corriger le problème lié au changement de domaine d'horloge, vous pourrez vous aider du lien <https://nandland.com/lesson-14-crossing-clock-domains/>.

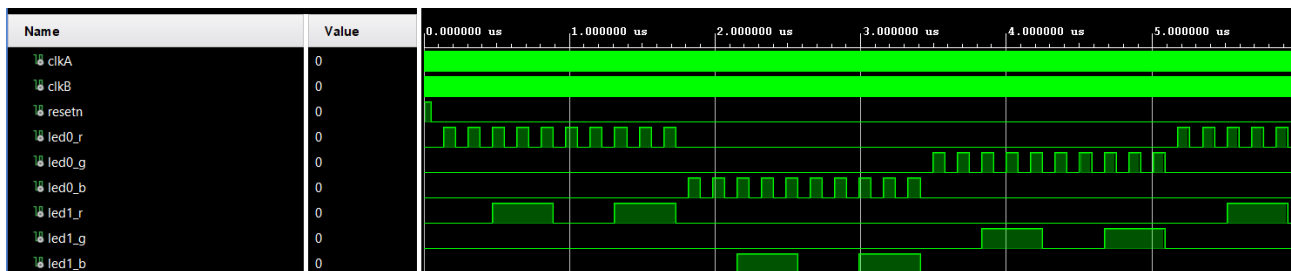
La solution (recommandée par nandland pour ce cas précis) : synchronisation par bascule :

-Côté source (clkA) : au lieu de transmettre l'impulsion telle quelle, on la convertit en un niveau logique stable : un bit qui bascule (toggle) à chaque impulsion update_s. Ce niveau reste stable jusqu'à la prochaine impulsion, donc n'importe quel front de clkB peut le capturer correctement, quelle que soit la différence de fréquence.

-Côté destination (clkB) : on fait passer ce bit à travers une chaîne de 2 à 3 bascules (2-FF ou 3-FF synchronizer) pour laisser le temps à une éventuelle métastabilité de se résorber.

-On détecte le changement (front) de ce bit synchronisé pour régénérer une impulsion propre d'un seul coup d'horloge clkB.

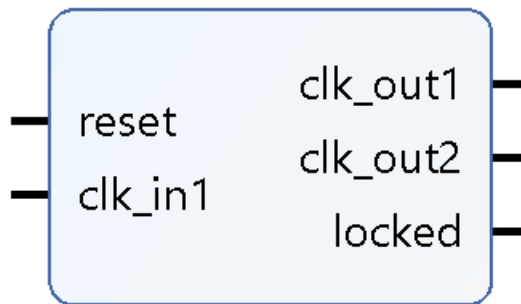
8. Mettez en place votre solution et testez-la en simulation. Si votre résultat de simulation n'est toujours pas valide, proposez une autre solution



Nous avons cette fois-ci les led0 et led1 bien synchronisées. Il n'y a pas de chevauchement où deux LEDs de couleur différentes clignotent en même temps. Les led0 et led1 clignotent avec la même couleur.

9. Pour générer plusieurs horloges vous aurez besoin d'une PLL. Trouvez quel est le nom de l'IP PLL de l'IP Catalog de Vivado puis ajoutez là à votre architecture. La fréquence de l'horloge en entrée de la PLL est de 100MHz. Les deux horloges en sortie doivent être à 50MHz et 250MHz.

Le nom de l'IP PLL dans Vivado : c'est le « Clocking Wizard » Je le configure avec une horloge d'entrée à 100 MHz et deux sorties à 50 MHz et 250 MHz comme suit :



	Input Clock	Port Name	Input Frequency(MHz)
	Primary	clk_in1	100.000

Output Clock	Port Name	Output Freq (MHz)	
		Requested	
☒ clk_out1	clk_out1	250.000	
☒ clk_out2	clk_out2	50.000	

```

component clk_wiz_0 is
port (
    clk_in1 : in  std_logic;
    reset   : in  std_logic;
    clk_out1 : out std_logic;
    clk_out2 : out std_logic;
    locked   : out std_logic
);
end component;

```

10. Effectuez la synthèse et placer des sondes (ILA) sur les signaux pertinents pour vérifier que le changement de domaine d'horloge s'est correctement passé.

```
-----
Start RTL Component Statistics
-----
Detailed RTL Component Info :
+---Adders :
      2 Input    28 Bit      Adders := 2
      2 Input     4 Bit      Adders := 1
      2 Input     1 Bit      Adders := 2
+---XORs :
      2 Input      1 Bit      XORs := 1
+---Registers :
           28 Bit    Registers := 2
           4 Bit     Registers := 1
           2 Bit     Registers := 4
           1 Bit     Registers := 17
+---Muxes :
      2 Input     4 Bit      Muxes := 1
      3 Input     2 Bit      Muxes := 1
      4 Input     2 Bit      Muxes := 2
      2 Input     1 Bit      Muxes := 11
      4 Input     1 Bit      Muxes := 2
-----
Finished RTL Component Statistics
-----
```

12. Effectuez le placement routage et étudiez les rapports générés.

Design Timing Summary			
Setup		Hold	Pulse Width
Worst Negative Slack (WNS):	0,660 ns	Worst Hold Slack (WHS):	0,122 ns
Total Negative Slack (TNS):	0,000 ns	Total Hold Slack (THS):	0,000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	90	Total Number of Endpoints:	94
All user specified timing constraints are met.			

$$WNS = 0,660\text{ns} > 0$$

Les contraintes de temps sont respectées